

Neon-Net

****A real aquarium fish net that catches AR jellyfish.**** Swing a physical net through your living room and a shoal of bioluminescent jellyfish, rendered through Snap Spectacles, scatters away from you or gets caught in it. An Arduino UNO Q on the handle reads the swing, predicts where it is going before it gets there, and decides how hard the game should be.

Built for the **Invent the Future with Arduino UNO Q and App Lab** hackathon, out of a long-standing wish to actually go jellyfishing rather than watch somebody do it in a cartoon. See STORY.md.

What it actually is

A mixed-reality game in which the controller is an aquarium fish net, the kind used to scoop fish out of a tank, and the AI runs on the net itself.

- ****The net is the input.**** An MPU-6050 on the handle samples acceleration at ~200 Hz on the UNO Q's STM32 side. There is no button and no hand tracking.
- ****The net suits the quarry.**** A fish net is the tool you would actually reach for to lift something soft out of water, which is exactly what a jellyfish is. The prop explains the game before anyone says a word.
- ****How hard you swing matters.**** A gentle scoop has a short reach but disturbs almost nothing. A committed lunge reaches twice as far but wakes every creature nearby, and a woken creature cannot be caught.
- ****The net answers back.**** An ERM motor gives four distinguishable haptic patterns: something nearby, something rare, a catch, a chain.
- ****The creatures have opinions.**** Two shoals group tight and back off when you land several catches quickly, and disperse and come closer when you keep missing.
- ****Two AI models run on the board****, not in the cloud, and both are visible on screen while you play.
- ****Two people can fish the same shoal.**** In shared play the UNO Q stops being a sensor with AI attached and becomes the game server: it owns every creature, runs their behaviour, and rules on who caught what. Two players lunging at the same jellyfish cannot both score it.

Nothing is sent to the internet. Everything runs on the board and the glasses over your local network.

The two AI models

Both run on the UNO Q's Qualcomm Linux side, in App Lab, in pure Python.

1. Predictive Trajectory AI

Fits a parabola through three smoothed acceleration samples and solves for its vertex, which is the peak of the swing. When the peak lands inside a 100 ms horizon, the swing is announced ****before it happens****, so the glasses can react while the net is still moving. That is what hides the wireless round trip.

A parabola rather than plain Newtonian extrapolation on purpose: a swing

decelerates as it approaches its peak, so projecting the current slope forward overshoots by roughly a factor of two.

Measured lead: **35 to 84 ms** ahead of the real peak.

2. Adaptive Difficulty AI

Watches each player's catch rate over a sliding ten-swing window and moves a single difficulty value to hold them near a 55 per cent catch rate. The Lens receives derived multipliers rather than the raw number, so all the balancing lives on the board.

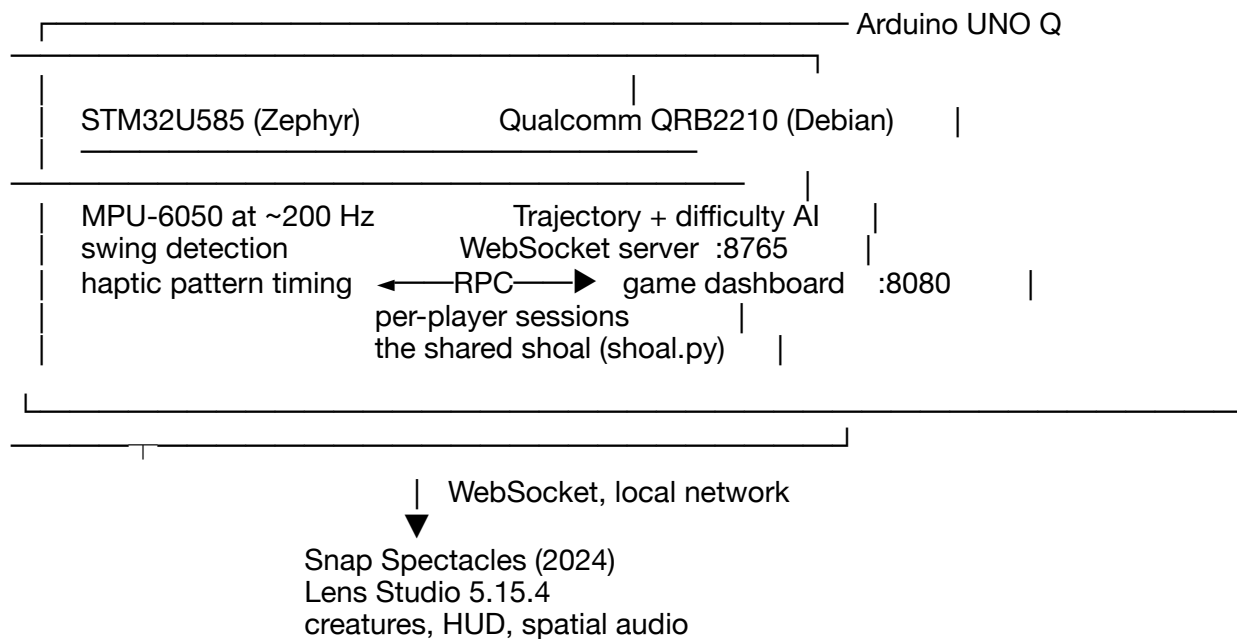
At high difficulty the rare creatures begin **cloaking**, fading down towards invisibility on the additive display.

Both models are heuristic by design. They are cheap enough for the poll loop, and unlike a trained network their behaviour is inspectable and repeatable, which matters when the whole thing has to survive being filmed in one take.

Architecture

The split is by latency budget, which is the only justification that matters.

...



...

Microsecond-sensitive work stays on the microcontroller: sampling the IMU and timing vibration patterns. Everything that benefits from a real OS lives on the Linux side: the models, the network, the dashboard. `Arduino\_RouterBridge` carries RPC between them.

Solo and shared play

The start screen offers two games, and what runs where depends on which you pick.

```
| | SOLO | SHARED |
|---|---|---|
| Creature positions and behaviour | Lens | **UNO Q** |
| Who caught what | Lens | **UNO Q**, first claim wins |
| Swing prediction, difficulty | UNO Q | UNO Q |
| Rendering, HUD, haptics | Lens | Lens |
```

In shared play the headsets render what they are told and *ask* to catch something: a Lens sends ``claim`` and the board grants it or refuses it. The board broadcasts the shoal at 15 Hz and each Lens reports its head pose at 10 Hz so creatures know when to bolt.

****On colocation, honestly.**** Two Spectacles do not share a coordinate origin. Each headset pins the board's room frame to wherever it stood when the round began, so two players who start from roughly the same spot facing the same way will agree about where the creatures are. What is exact is the ****state****: the same creatures, the same ids, one ruling on each catch. True shared spatial anchors need Connected Lenses, which this deliberately does not use.

Repository layout

```
| Path | What it is |
|---|---|
| `sketch.ino` | MCU firmware. IMU sampling, swing detection, haptic patterns, watchdog |
| `main-nodeps.py` | Linux side. WebSocket server, multiplayer sessions, AI glue |
| `main-nodeps-standalone.py` | **The file to paste into App Lab.** Generated, single file, no imports |
| `build-standalone.py` | Generates the above by inlining the three modules |
| `neon_ai.py` | Both AI models. No hardware imports, so it runs on a laptop |
| `shoal.py` | The authoritative shared shoal, simulated on the board |
| `test_shoal.py` | Tests for the shared shoal, including the multiplayer rules |
| `dashboard.py` | The game dashboard served on port 8080 |
| `test_neon_ai.py` | Tests for both models against synthetic IMU traces. `python3 test_neon_ai.py` |
| `mock-net.py` | Stands in for the whole board so the Lens can be developed without hardware |
| `make-audio.py` | Synthesises both sound files from scratch, standard library only |
| `Lumicatch/` | The Lens Studio project |
| `Lumicatch/Assets/LumiCatchManager.ts` | The Lens. Creature AI, HUD, capture, networking |
| `diagnostics/` | Standalone test sketches: IMU, motor, I2C scan |
| `LumiCatch-Build-Guide` | The full build log, including everything that went wrong |
| `BOM.md` | Bill of materials |
| `wiring-diagram.png` | Circuit diagram |
| `breadboard-view.png` | Breadboard view: jumper colours and hole positions |
```

Build it

Full instructions, with the failure modes, are in
LumiCatch-Build-Guide.md. In outline:

1. **Wire the IMU and the motor** — wiring-diagram.png. Do not omit the flyback diode.
2. **Check the hardware** — flash `diagnostics/phase1-imu-test.ino`` and `diagnostics/phase2-motor-test.ino`` before going near the game code.
3. **Flash the firmware** — paste `sketch.ino`` into your App Lab sketch.
4. **Run the Linux side** — paste `main-nodeps-standalone.py`` over the App's `main.py``. Make sure `app.yaml`` publishes ports: `ports: [8765, 8080]`. An empty `ports: []` means a perfectly working server that nothing can reach.
5. **Open the dashboard** at `http://<board-ip>:8080``.
6. **Point the Lens at the board** — set `serverUrl`` to `ws://<board-ip>:8765`` and untick `simulate``.
7. **Mount it on the net**, power bank in your pocket, and re-measure your swing thresholds.

No hardware yet?

`mock-net.py`` impersonates the entire board, including both real AI models, and serves the same dashboard:

```
``sh
python3 mock-net.py    # Enter = swing, a number = swing with that peak
``
```

Five things that will bite you

Each of these cost real hours. They are written up properly in the build guide.

1. **`Serial`` prints nothing.** Including `Arduino_RouterBridge.h`` replaces it. Use `Monitor``, and give it `delay(3000)`` after `begin()``.
2. **`Bridge.begin()`` after `Monitor.begin()`` silently kills all printing.** Initialise Bridge first. They share a transport.
3. **`ports: []` in `app.yaml``** means the container never publishes. SSH works, your server is unreachable, and nothing logs an error.
4. **A brownout resets the MPU-6050 into sleep**, where it still acknowledges on I2C and returns nothing but zeros. Looks exactly like a dead sensor. Hence the flyback diode and a watchdog that re-initialises it.
5. **Lens Studio imports this glTF animation with the wrong duration.** `clip.end`` came in as 0.1375 s instead of 4.125 — the keyframe times, specified in seconds, read as frames and divided by 30. The creature looked completely static. If an imported animation looks frozen, print `clip.end`` before anything else.

Licence and attribution

Code in this repository is available under the MIT licence. All eight sound files are synthesised from scratch by make-audio.py and are covered by it too.

The two jellyfish models are third-party, both under

[CC BY 4.0](https://creativecommons.org/licenses/by/4.0/), which permits commercial use and derivatives on condition of attribution:

- **'Simple Jellyfish'** by **RickStikkelorum** — [source](https://sketchfab.com/3d-models/simple-jellyfish-f77876d8297846eeb23c4ad82dbebb97), [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)
- **'Simple Spotted Jellyfish (Baked Animation)'** by **n-** — [source](https://sketchfab.com/3d-models/simple-spotted-jellyfish-baked-animation-d5006697ad3c4bc1ac814110cde19af2), [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)

Neither model file has been altered. The Lens replaces their materials and adjusts scale and clip length at runtime; no geometry, rigging or animation data was edited. See BOM.md for the full note.